

Data Analytics Capstone

Evaluating the Business Impact of Artificial Intelligence Adoption Across the Quality Engineering Lifecycle

Interim Report

Satish Venugopal

Walsh College

QM 640 – Data & Analytics – Capstone Project

Mentor: Dr. Sanhita Karmakar

14-July-2026

GitHub Repository and Reproducibility Materials

Repository URL: https://github.com/satisho/QM640_Data-Analytics_Capstone

The repository will provide the evaluator with the raw dataset, cleaned analytical datasets, executed notebook, supporting code, model outputs, and report evidence. The exact repository address was not included in the supplied final-submission package, so the highlighted placeholder above must be replaced with the active link before the report is submitted.

```
QM640_EQEPD_Interim_GitHub_Repository/
├── data/
│   ├── raw/
│   │   └── EQEPD_Dataset_v5.xlsx
│   └── processed/
│       ├── EQEPD_Cleaned_Project_Level.csv
│       └── EQEPD_Engineered_Phase_Level.csv
├── notebooks/
│   ├── QM640_EQEPD_Interim_Analysis.ipynb
│   └── QM640_EQEPD_Interim_Analysis.html
├── outputs/
│   ├── figures/
│   └── tables/
├── reports/
│   ├── QM640_EQEPD_Interim_Report_Satish_Venugopal.docx
│   └── Interim_Report_Figure_Guide.md
├── scripts/
│   ├── run_notebook.bat
│   └── run_notebook.sh
├── .gitignore
├── README.md
└── requirements.txt
```

Figure 1: *Proposed Repository Folder Structure*

Note. The folder structure is aligned with the files named in the capstone submission package. The active repository URL must be inserted before submission

Planned repository paths are data/raw/EQEPD_Dataset_v5.xlsx for the source workbook, data/processed/ for the cleaned project-level and engineered phase-level files, notebooks/ for the executed Jupyter notebook, outputs/ for statistical and model results, and reports/ for the interim and final reports. This structure separates immutable raw data from processed data and analysis artefacts, making it easier for an evaluator to reproduce the work without modifying the source file.

Introduction

Background and Context

Artificial Intelligence (AI) is increasingly embedded in the software delivery lifecycle, where it is used to generate requirements-based tests, suggest test scenarios, create automation code, prioritise regression suites, classify defects, and support release decisions. Within enterprise Quality Engineering (QE), these capabilities are attractive because testing frequently consumes a substantial share of delivery effort and is often constrained by short release cycles, large regression inventories, and uneven access to specialised testing skills. The operational promise is straightforward: if AI can reduce the effort required to plan, design, automate, execute, and analyse tests without lowering quality, organisations may release software faster and redirect skilled engineers toward higher-value work. The management challenge is that this promise is often communicated through tool demonstrations or isolated pilots rather than through consistent project-level evidence.

The setting for this capstone is an enterprise QE portfolio represented by the Enterprise Quality Engineering Performance Dataset (EQEPD). The dataset records project, application, team, sprint, complexity, workload, AI-use, baseline-effort, and actual-effort information across six lifecycle phases: Test Planning, Test Design, Automation Development, Test Execution, Regression Testing, and Defect Management. The principal stakeholders are QE leaders who decide where to invest, delivery managers who plan effort and staffing, test architects who select suitable use cases, and project teams whose operating practices influence whether AI assistance produces sustainable value. For these stakeholders, a phase-level view is important because AI may create strong benefits in repetitive and information-intensive activities while producing

smaller gains in activities that depend heavily on judgement, environment stability, or physical execution.

Prior research demonstrates that machine learning and large language models can support several testing activities, including test generation, test prioritisation, and test planning, but it also identifies reliability, maintainability, hallucination, and human-oversight concerns (Durelli et al., 2019; Halder & Capretz, 2026; Wang et al., 2024). This makes empirical measurement essential. A credible adoption decision should distinguish observed effort reduction from perceived convenience, compare AI-assisted work with a meaningful traditional baseline, and account for project context. It should also avoid treating all lifecycle phases as equivalent, because the economics and risks of automating regression selection differ from those of generating a test strategy or deciding defect severity.

This interim report presents the project as a standalone study. It explains the research problem, literature foundation, data structure, cleaning decisions, exploratory findings, modelling plan, and preliminary results. The analytical work is substantially advanced: the raw data has been profiled and cleaned, project-level records have been transformed into phase-level observations, exploratory data analysis has been completed, initial statistical tests have been run, and three predictive models have been compared. Remaining work is focused on final sensitivity checks, presentation of business recommendations, repository publication, and final report packaging.

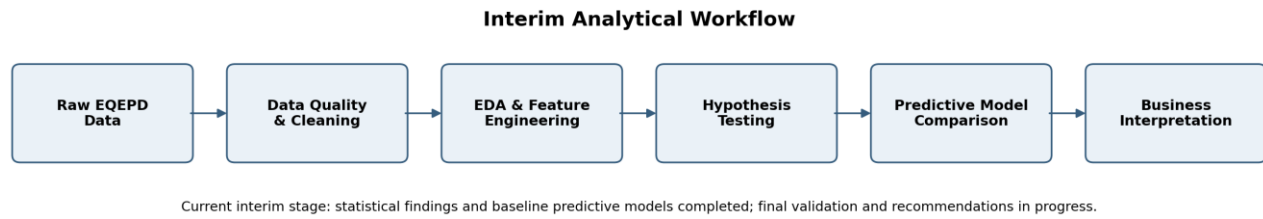


Figure 2: *Interim Analytical Workflow*

Note. The workflow links data preparation, explanatory analysis, statistical testing, predictive modelling, and business interpretation. At the interim stage, the first five analytical steps have preliminary outputs.

Problem Statement

Organisations are investing in AI-enabled QE, but they often lack comparable operational evidence showing whether AI-assisted activities deliver higher productivity than traditional activities, which lifecycle phases generate the strongest value, and which project characteristics influence the outcome. Without this evidence, investment may be spread uniformly across unsuitable use cases, pilot results may be overgeneralised, and expected savings may be based on vendor claims rather than observed effort. This project addresses the gap by creating consistent productivity measures from baseline and actual effort, comparing AI-assisted and traditional phase observations, examining phase-level differences, controlling for project context, and testing whether machine-learning models can provide useful directional estimates for future prioritisation.

Purpose of the Study

The purpose of the study is to explain and predict productivity improvement associated with AI adoption across the QE lifecycle. The explanatory component evaluates whether AI usage is associated with a statistically and practically meaningful reduction in effort and whether results differ by lifecycle phase. The predictive component compares supervised regression models that estimate overall project/sprint productivity improvement from project context, baseline workload, and AI-adoption features. The intended output is not an autonomous investment engine; it is an evidence-based decision-support framework that helps QE leaders identify stronger candidate phases, recognise uncertainty, and define the additional measures needed for a complete business case.

Interim Project Status (Progress Snapshot)

Status	Work Items
Completed	Raw workbook inventory; data-type and missing-value profiling; exact duplicate removal; categorical standardisation; project-level and phase-level dataset construction; initial exploratory data analysis; RQ1 and RQ2 hypothesis tests; preliminary regression analysis; baseline model comparison.
In progress	Robustness review of statistical assumptions; grouped cross-validation interpretation; feature-importance stability review; integration of findings by research question; APA 7 formatting; preparation of reproducibility materials and narrative evidence.
Pending	Insert active GitHub URL; complete final sensitivity checks; finalise business recommendations and adoption roadmap; validate the final notebook execution from a clean environment; publish the final submission package.

Table 1: *Interim Project Status*

Note. Status reflects the analytical work represented in the supplied capstone package while preserving an interim distinction between preliminary evidence and final conclusions.

Scope and Objectives

Research Problems Identified

- Measurement inconsistency. AI benefits are commonly described through time saved, productivity, quality improvement, or user perception, but these measures are not always defined consistently. This project uses baseline and actual effort to create a common phase-level productivity measure.
- Lifecycle heterogeneity. The suitability of AI differs across Test Planning, Test Design, Automation Development, Test Execution, Regression Testing, and Defect Management. A portfolio average can conceal phases that are ready to scale and phases that require controlled pilots.
- Context and confounding. Teams that adopt AI may also differ in application maturity, complexity, workload, leadership support, and delivery period. The analysis therefore examines project characteristics and uses regression controls rather than attributing every observed difference directly to AI.
- Decision-support uncertainty. Even when statistical relationships are strong, project-level prediction may remain imperfect. The study evaluates whether models are sufficiently accurate for directional prioritisation and explicitly defines the boundary between model support and managerial judgement.

Research Objectives

- Evaluate the relationship between AI adoption and productivity improvement across selected QE activities.

- Compare AI-assisted and traditional approaches using consistently calculated effort-based metrics.
- Identify the lifecycle phases that produce the highest mean improvement and cumulative effort savings.
- Determine which project and workload characteristics are associated with productivity improvement after statistical controls.
- Compare interpretable and nonlinear regression models using leakage-aware grouped validation.
- Translate preliminary evidence into a phased adoption and measurement approach for enterprise QE decision-makers.

Research Questions and Hypotheses

Question	Research Focus	Null Hypothesis	Alternative Hypothesis
RQ1	Does AI adoption significantly improve productivity across QE activities?	H ₀₁ : No significant difference between AI-assisted and traditional activities.	H ₁₁ : AI-assisted activities show significantly higher productivity improvement.
RQ2	Does AI-generated business value differ across QE lifecycle phases?	H ₀₂ : Productivity improvement does not differ across phases.	H ₁₂ : One or more phases differ significantly.
RQ3	Which project characteristics and QE factors influence productivity improvement?	H ₀₃ : No project or QE factor has a significant relationship with improvement.	H ₁₃ : One or more factors have a significant relationship.
RQ4	Can machine-learning models predict productivity improvement and identify priority AI-adoption areas?	H ₀₄ : Models do not provide useful predictive or prioritisation value.	H ₁₄ : Models provide useful directional prediction and prioritisation.

Table 2: Research Questions and Hypotheses

Note. A 95% confidence level and significance threshold of $\alpha = .05$ are used for the inferential analyses

Study Scope

The study includes anonymised project/application/team observations from 2023 through 2026 and covers six QE phases. It analyses binary AI-use indicators, project complexity, user-story volume, test-case volume, baseline effort, and actual effort. The primary definition of business value is effort reduction expressed as hours saved and percentage productivity improvement. Direct financial cost, licence cost, defect leakage, automation coverage, test coverage, AI tool identity, AI-assistance intensity, and human-review effort are not available in the current dataset. Consequently, the interim findings should be interpreted as evidence about operational productivity rather than a complete return-on-investment calculation.

Literature Survey

Literature Review Approach

The literature review was designed to support all four research questions rather than provide a broad history of AI. Searches focused on peer-reviewed journal articles, conference papers, systematic reviews, and established statistical-learning references. Sources were identified through IEEE Xplore, SpringerLink, ScienceDirect, ACM-related indexes, and scholarly metadata services using combinations of the terms artificial intelligence, machine learning, large language models, software testing, quality engineering, test generation, regression testing, test prioritisation, productivity, Random Forest, Gradient Boosting, mean absolute error, and root mean square error. Priority was given to studies published from 2019 through 2026 for the domain review, with older foundational sources retained where they define widely used methods. Inclusion criteria required a direct connection to software testing or QE, an empirical or structured-review method, clear findings relevant to productivity, test effectiveness, adoption risk, or model selection, and sufficient bibliographic information for APA 7 citation. Purely promotional tool descriptions, articles without identifiable methods, and studies focused only on testing AI systems rather than using AI to improve testing were excluded unless they offered a transferable governance insight. The final set combines domain evidence with methodological foundations so that model choice and evaluation decisions are linked to both the software-testing problem and accepted data-science practice.

Summary of Key Literature

Author (Year)	Domain/Context	Dataset/Setting	Method(s)	Key Findings	How It Supports RQ(s)/Decisions
Durelli et al. (2019)	Machine learning in software testing	Systematic mapping of peer-reviewed studies	Mapped ML techniques and testing activities	ML was concentrated in test generation, defect prediction, and related automation tasks; industrial evidence was less mature.	Supports the lifecycle framing and the need for operational evidence across RQ1–RQ3.
Wang et al. (2024)	Large language models in software testing	Survey of LLM-based testing research	Taxonomy and research landscape	LLMs can support test generation and related tasks, but reliability, evaluation, and human oversight remain central concerns.	Supports AI-use cases and the governed interpretation of RQ1, RQ2, and RQ4.
Haldar & Capretz (2026)	Automated test-plan generation	Three open-source web applications; five LLMs	Empirical generation and standards-based evaluation	Generated plans showed useful potential but variable quality; human validation remained necessary.	Explains why Test Planning may show lower or less stable gains and informs RQ2.
Ouédraogo et al. (2026)	LLM unit-test generation	216,300 tests across Defects4J, SF110, and CMD	Large-scale comparison of four models and five prompting methods	Reasoning prompts improved structural quality, yet hallucination and compilation failures remained; hybrid validation was recommended.	Supports feature/use-case specificity and human review for RQ2 and RQ4.
Ardic et al. (2026)	Human–AI interaction in testing	Observational study with 12 students	Workflow observation and outcome assessment	Participants reported time savings and lower cognitive load, alongside trust, quality, and ownership concerns.	Provides a human-factors explanation for variation not captured by the dataset; relevant to RQ3.
Romero et al. (2026)	Test-case prioritisation	Evolving software builds and test histories	AutoML with grammar-guided genetic programming	Adaptive pipelines reduced the ML expertise required and adjusted to changes across builds.	Supports Regression Testing as a high-value AI area and motivates nonlinear models for RQ2/RQ4.

Author (Year)	Domain/Context	Dataset/Setting	Method(s)	Key Findings	How It Supports RQ(s)/Decisions
Yoo & Harman (2012)	Regression test optimisation	Survey of minimisation, selection, and prioritisation methods	Structured literature survey	Regression optimisation can reduce testing cost and improve early fault detection under time constraints.	Provides theoretical support for the strong Regression Testing results in RQ2.
Garousi et al. (2020)	Industry testing challenges	Empirical industry study	Practitioner evidence and challenge analysis	Testing outcomes depend on organisational, process, tooling, and skill constraints rather than technique alone.	Supports contextual controls and cautions against causal overstatement in RQ3.
Raharjana et al. (2025)	Deep learning in software testing	737 Scopus-indexed publications	Bibliometric analysis	Research increased sharply after 2017, led by defect prediction, code analysis, and automated testing, with important gaps remaining.	Establishes the maturity and uneven coverage of the field for RQ1–RQ3.
Breiman (2001)	Random Forest methodology	General supervised-learning setting	Ensemble of decorrelated decision trees	Random Forests model nonlinear relationships and interactions while reducing the instability of individual trees.	Justifies a robust nonlinear model for RQ4.
Friedman (2001)	Gradient Boosting methodology	General regression and classification setting	Sequential additive modelling	Gradient boosting improves predictions by iteratively fitting residual error and can capture complex tabular patterns.	Justifies a second nonlinear benchmark for RQ4.
Willmott & Matsuura (2005)	Model error evaluation	Comparative discussion of error metrics	Analytical comparison of MAE and RMSE	MAE is directly interpretable as average error; RMSE gives greater weight to larger misses.	Supports reporting multiple complementary performance metrics for RQ4.

Table 3: *Literature Relevance Matrix*

Note. The matrix summarises 12 directly relevant sources. Additional statistical references are included in the bibliography and used in the methodology discussion.

Thematic Synthesis of the Literature

AI Value Is Activity-Specific

The literature consistently shows that “AI in testing” is not a single intervention. Durelli et al. (2019) mapped machine-learning applications across multiple activities, while Wang et al. (2024) documented a rapidly expanding landscape of large-language-model use cases. The strongest evidence tends to appear where historical data, repetitive decisions, or structured artefacts are available—for example, test generation, defect prediction, and regression prioritisation. This pattern supports the project’s decision to analyse six phases separately. A single portfolio average would be insufficient because the input artefacts, human judgement requirements, and cost of an incorrect output differ by phase. The literature therefore provides a conceptual basis for RQ2 and for reporting both percentage improvement and total effort saved rather than treating adoption as uniformly beneficial.

Productivity Gains Require Validation and Human Oversight

Recent empirical work on large language models provides a balanced view of productivity. Haldar and Capretz (2026) found that LLM-generated test plans could improve automation and efficiency, but plan quality varied and still required human validation. Ouédraogo et al. (2026) observed improved readability and structure under stronger prompting strategies, yet also reported persistent hallucination-related failures and high compilation-failure rates in some settings. Ardic et al. (2026) found perceived time-saving and cognitive-load benefits alongside concerns about trust, quality, and ownership. Together, these findings justify measuring effort reduction while preserving explicit governance boundaries. In this capstone, productivity improvement is not treated as proof that output quality is automatically acceptable; it is treated as

one dimension of business value that must ultimately be combined with quality and risk measures.

Regression Testing Is a Strong Candidate for Data-Driven Optimisation

Regression Testing has an established history of algorithmic optimisation. Yoo and Harman (2012) showed that test minimisation, selection, and prioritisation address the cost and timing constraints of repeatedly executing large suites. Romero et al. (2026) extended this logic by demonstrating that automated machine learning can adapt test-prioritisation pipelines as software builds evolve. This evidence is directly pertinent to the EQEPD results because Regression Testing is both repetitive and historically measurable, making it well suited to data-driven selection and prioritisation. The literature does not guarantee that every regression context will achieve the same benefit, but it makes a higher preliminary productivity result in this phase theoretically plausible and provides a rationale for prioritising it in the phase comparison.

Model Performance Must Be Interpretable in Operational Terms

The methodological literature supports comparing a transparent linear baseline with nonlinear ensemble models. Linear Regression provides coefficients that help explain direction and magnitude, while Random Forest and Gradient Boosting can represent interactions and nonlinear relationships that are common in project data (Breiman, 2001; Friedman, 2001; James et al., 2021). Reporting MAE, RMSE, and R^2 avoids relying on a single performance statistic. MAE communicates average error in percentage points, RMSE highlights occasional large misses, and R^2 describes the proportion of variation explained on unseen data (Willmott & Matsuura, 2005). This combination aligns the model evaluation with RQ4: the question is not merely which algorithm ranks first, but whether the resulting error is acceptable for directional portfolio screening and whether the model's limitations are clearly disclosed.

Data Description

Data Source(s) and Access

The analysis uses the Enterprise Quality Engineering Performance Dataset (EQEPD), an anonymised and aggregated academic dataset prepared for this capstone (Venugopal, 2026). The dataset does not contain client names, employee identifiers, source code, production data, or personally identifiable information. The evaluator will access the source workbook and processed files through the GitHub repository identified on page 2. The raw workbook should remain unchanged in the data/raw directory, while cleaned and engineered outputs should be stored under data/processed so that transformations are transparent and reproducible.

The source is cited as an unpublished data set because it is project-specific rather than downloaded from a public statistical portal. Once the repository is active, the bibliography entry and the GitHub page should be updated with the permanent URL. The repository is also expected to contain the executed notebook and exported HTML report so that tables, plots, statistical tests, and model results can be inspected independently of this narrative report.

Dataset Overview

Dataset Overview

Dataset Measure	Value
Raw records (rows)	459
Raw variables (columns)	24
Time period	2023–2026
Unit of analysis	Project/application/team observation for a sprint
Exact duplicates identified	2
Clean project/sprint observations	457

Dataset Measure	Value
Potential phase observations	2,742 (457×6 phases)
Valid phase productivity observations	2,705
AI-assisted valid phase observations	906
Traditional valid phase observations	1,799
Applications after standardisation	8
Teams after standardisation	8
Primary phase target	Productivity Improvement % = $(\text{Baseline} - \text{Actual}) / \text{Baseline} \times 100$
Primary project target	Overall Productivity Improvement % based on total baseline and actual effort

Note. Counts are based on the supplied analytical package. A phase observation is valid only when baseline and actual effort allow the productivity target to be calculated.

Each raw record contains six AI-use indicators, six baseline-effort fields, and six actual-effort fields in addition to project context. The wide raw structure is appropriate for operational data capture, but statistical comparison across phases is easier in a long phase-level structure. The transformation therefore creates one row per project/sprint/phase combination and carries the relevant AI flag, baseline effort, actual effort, and context variables into that row. This produces up to 2,742 phase observations before excluding cases where the productivity outcome cannot be calculated.

The phase-level target expresses the percentage by which actual effort is below or above baseline. Positive values indicate effort reduction, zero indicates performance exactly at baseline, and negative values indicate that actual effort exceeded the baseline. For example, a baseline of 20 hours and actual effort of 16 hours produces $(20 - 16) / 20 \times 100 = 20\%$ productivity

improvement. A baseline of 20 hours and actual effort of 22 hours produces –10%, indicating an overrun. The project-level target aggregates effort across all six phases before calculating the percentage, preventing a small phase from receiving the same weight as a large phase.

EQEPD Dataset Preview (Selected Columns)

Sprint	Application	Team	Complexity	Stories	Tests	AI-Test Design	AI-Regression	Baseline TD	Actual TD
2023-S01	Legacy Service Platform	Legacy Support QA	High	11.0	13.0	No	No	7.51	8.7
2023-S01	CRM Sales & Service	CRM Platform QA	High	nan	86.0	No	No	45.82	47.31
2023-S01	Billing & Payments	Billing & Payments QA	Medium	9.0	75.0	No	No	32.32	32.08
2023-S01	Customer Web Portal	EIP-QA	Low	12.0	57.0	No	No	20.8	25.6
2023-S01	ERP Operations	ERP Operations QA	Medium	9.0	78.0	No	No	33.67	35.65

Figure 3: EQEPD Dataset Preview

Note. Selected columns from the first five records are displayed for readability. The complete 24-column source workbook is to be made available in the GitHub repository.

Data Dictionary (Summary)

Variable Name	Definition	Datatype	Allowed Values/Range	Missing Values Handling	Notes
Sprint #	Delivery period identifier; source of year and sprint-number features	Text	YYYY-Snn	Not imputed	Grouping variable for validation
Application_Name	Anonymised application/platform category	Categorical	Standardized to 8 applications	Mode not used; category retained	One-hot encoded for models
Team_Name	Anonymised QE team category	Categorical	Standardised to 8 teams	Category retained	Potential organisational context
Complexity	Assessed project complexity	Ordinal category	Low, Medium, High	Standardised aliases	One-hot encoded
User story count	User stories in the observation	Numeric count	Non-negative	Median imputation	Workload proxy

Variable Name	Definition	Datatype	Allowed Values/Range	Missing Values Handling	Notes
test case count	Test cases in the observation	Numeric count	Non-negative	Median imputation	Testing-intensity proxy
AI Usage Testing Phase (...)	AI use in each of six phases	Binary category	Yes/No	Standardised variants	Converted to 1/0 flags
Baseline effort (...)	Expected effort hours for each phase	Numeric continuous	Greater than 0 for target calculation	Not imputed for outcomes	Primary denominator
Actual effort (...)	Observed effort hours for each phase	Numeric continuous	Non-negative	Not imputed for outcomes	Primary observed effort
QE_Phase	Lifecycle phase in long dataset	Categorical	Six named phases	Derived	Used in phase comparison
Effort_Saved	Baseline effort minus actual effort	Numeric continuous	Positive or negative	Derived only when both efforts valid	Absolute operational benefit
Productivity_Improvement_Pct	Relative phase effort reduction	Numeric continuous	Unbounded in raw calculation	No imputation; winsorised only for modelling	Primary RQ1–RQ3 outcome
AI_Phase_Count	Number of phases using AI	Integer	0–6	Derived	Adoption breadth
AI_Adoption_Rate	AI_Phase_Count divided by 6	Numeric proportion	0–1	Derived	Normalised adoption breadth
Overall_Productivity_Improvement_Pct	Aggregate project/sprint effort reduction	Numeric continuous	Unbounded in raw calculation	Winsorised at 1st/99th percentiles for modelling	RQ4 target
AI_Exposure by Phase	AI flag multiplied by phase baseline share	Numeric proportion	0–1	Derived	Workload-weighted AI use

Table 5: Data Dictionary Summary

Note. The complete field-level dictionary, including all six baseline, actual-effort, and AI-use fields, is provided in Appendix B.

GitHub Data Availability Statement

The raw and processed datasets will be available in the GitHub repository under the following paths:

- Raw data: data/raw/EQEPD_Dataset_v5.xlsx;
- processed project data: data/processed/EQEPD_Cleaned_Project_Level.csv;
- processed phase data: data/processed/EQEPD_Engineered_Phase_Level.csv;
- code and notebooks: notebooks/QM640_EQEPD_Capstone_Final.ipynb;
- model outputs: outputs/.

The repository should also include a README that states the Python environment, package versions, execution order, target definitions, and any ethical or confidentiality constraints. The active URL is provided in page 2.

Analysis

Data Cleaning

Data cleaning was deliberately conservative so that the analysis would not remove operationally difficult cases merely because they were unusual. The raw workbook contained 459 records and 24 variables. Two rows were exact duplicates and were removed, leaving 457 project/sprint observations. Near-duplicate rows were not automatically discarded because repeated application/team/sprint combinations can represent legitimate records. The duplicate rule was therefore exact equality across all columns, which is reproducible and avoids subjective deletion.

Missingness was limited and dispersed. User story count had the highest missing count at six records (1.31%). Five values were missing in actual Regression Testing effort, five in actual

Automation Development effort, four in actual Test Design effort, and four in baseline Automation Development effort; other affected fields had between two and three missing values. Contextual count variables were median-imputed because they are predictors rather than outcomes and because median imputation is less sensitive to skew. Baseline and actual effort were not imputed for productivity calculations. Imputing these fields would manufacture the dependent variable and could create artificial evidence of improvement. Instead, only the affected phase observation was excluded when the target could not be computed.

Categorical inconsistencies were standardised before grouping. Application labels were consolidated from 15 raw variants to 8 standardised applications, and team labels were reduced from 13 variants to 8 teams. AI-use entries such as Yes, YES, Y, yes, No, NO, N, and no were mapped to consistent Yes/No values. Complexity abbreviations and case variants were mapped to Low, Medium, and High. Year and sprint number were parsed from Sprint # to support trend analysis and grouped validation. These transformations were documented rather than applied silently so that an evaluator can compare raw and cleaned categories.

Potential outliers were reviewed rather than removed. The phase-level productivity distribution produced 86 observations outside the conventional 1.5 interquartile-range bounds, with approximate bounds of -41.00% and 48.25% . Such values may represent genuine overruns, unusually successful automation, small denominators, or data-entry issues. They were retained for descriptive and non-parametric hypothesis testing because excluding them could suppress legitimate operational variability. For predictive modelling only, the project-level target was winsorised at the 1st and 99th percentiles to reduce the influence of extreme outcomes on model stability while preserving the records.

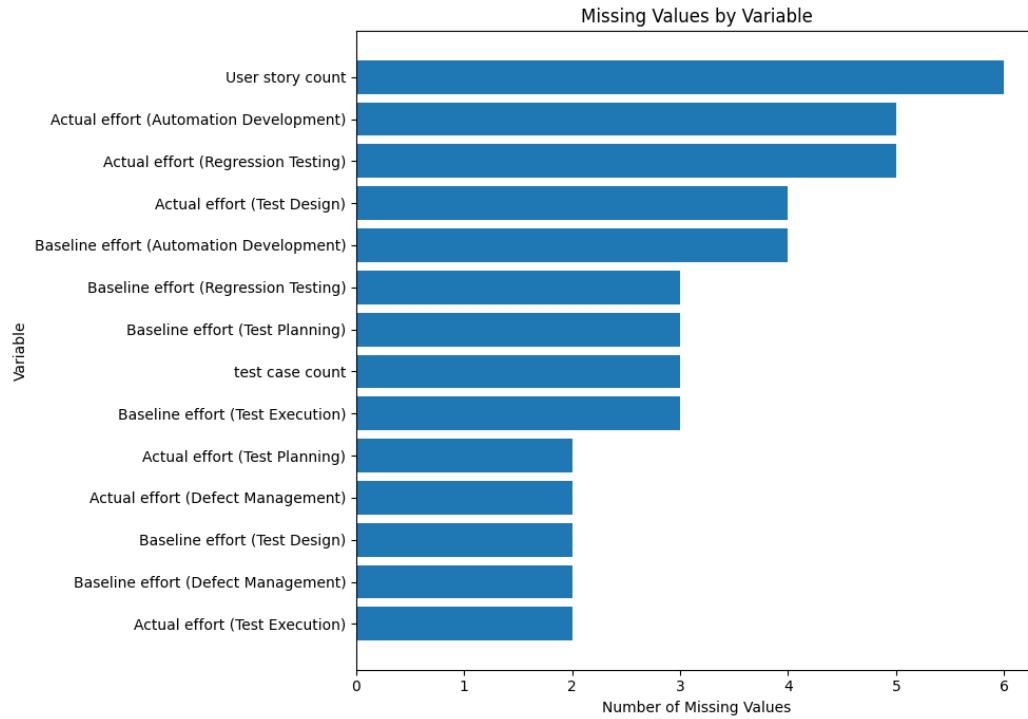


Figure 4: *Missing Values by Variable*

Note. Missingness was below 1.31% for every variable. Context counts were median-imputed, while effort fields were not imputed for outcome calculation.

Issue	Variables Affected	Detection Method	Treatment Applied	Rationale
Missing values	User story count, test case count, selected baseline/actual effort fields	Null counts and percentages	Median-impute contextual counts; do not impute effort outcomes; exclude only invalid phase target	Preserves predictor coverage without creating artificial productivity values
Exact duplicates	All 24 raw columns	Full-row equality check	Remove 2 exact duplicates	Avoids double-counting while retaining legitimate repeated contexts
Application aliases	Application_Name	Unique-value and frequency review	Consolidate 15 raw labels to 8 applications	Prevents fragmented group statistics

Issue	Variables Affected	Detection Method	Treatment Applied	Rationale
Team aliases	Team_Name	Unique-value and frequency review	Consolidate 13 raw labels to 8 teams	Creates consistent organisational categories
AI label variants	Six AI-use variables	Value-frequency review	Map Yes/YES/Y and No/NO/N variants to Yes/No	Required for valid binary comparison
Complexity variants	Complexity	Unique-value review	Map abbreviations and case variants to Low/Medium/High	Supports interpretable grouping and modelling
Outliers	Productivity improvement targets	Histogram, boxplot, IQR rule	Retain for EDA and hypothesis tests; winsorise project target only for models	Balances operational realism and model stability
Target leakage	Actual effort and derived outcome fields	Feature lineage review	Exclude actual effort, effort saved, and outcome-derived fields from predictive features	Prevents future information from inflating RQ4 performance

Table 6: Data Cleaning Log

Exploratory Data Analysis Results

AI Adoption Trend

AI adoption increased materially over the four-year period. The proportion of phase observations marked as AI-assisted rose from 7.17% in 2023 to 20.66% in 2024, 42.67% in 2025, and 64.20% in 2026. Figure 5 therefore describes a staged adoption journey rather than a single transition.

This matters for interpretation because later observations may also reflect improved tooling, user familiarity, process maturity, or portfolio selection. Year and sprint variables are retained in the influence model to reduce the risk that the AI coefficient merely captures a time trend.

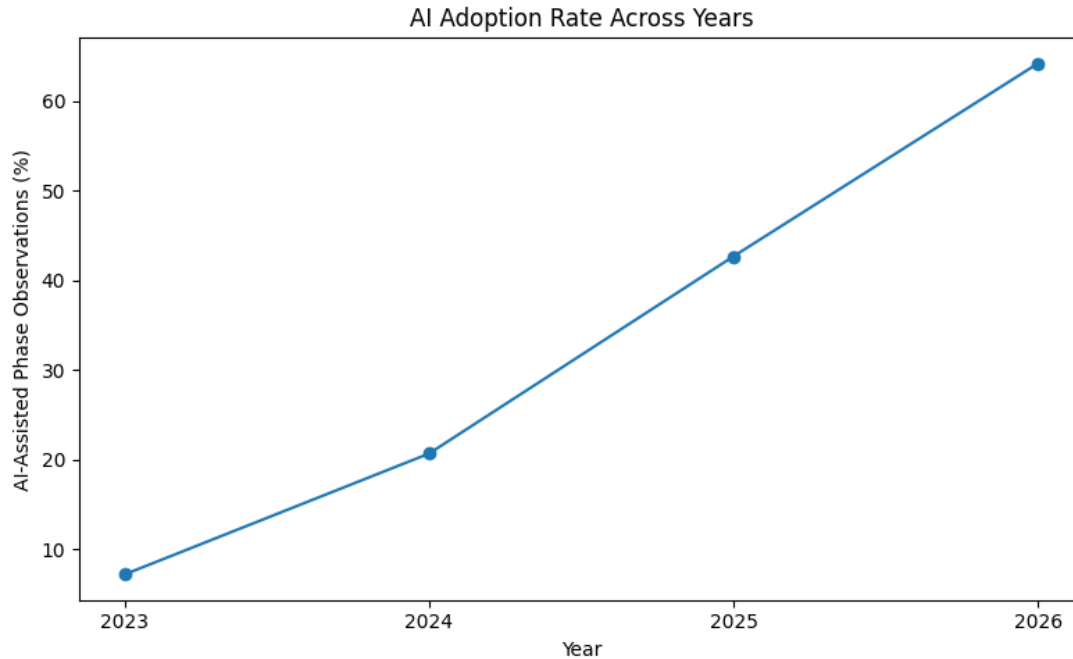


Figure 5: *AI Adoption Rate Across Years*

Note. The adoption rate is the percentage of phase observations with AI_Used = Yes in each year.

Adoption by QE Lifecycle Phase

Adoption was uneven across the lifecycle, as shown in Figure 6. Test Design had the highest AI-use rate at 66.08%, followed by Automation Development at 54.27% and Regression Testing at 38.29%. Defect Management was 24.95%, Test Planning was 13.13%, and Test Execution was only 6.13%. This distribution is operationally meaningful: phases with more structured digital artefacts and reusable patterns appear to have adopted AI earlier, whereas activities tied to environments, execution dependencies, and high-consequence judgement show lower penetration. The phase differences also mean that raw totals alone cannot be used to infer value; both adoption volume and phase-level improvement must be examined.

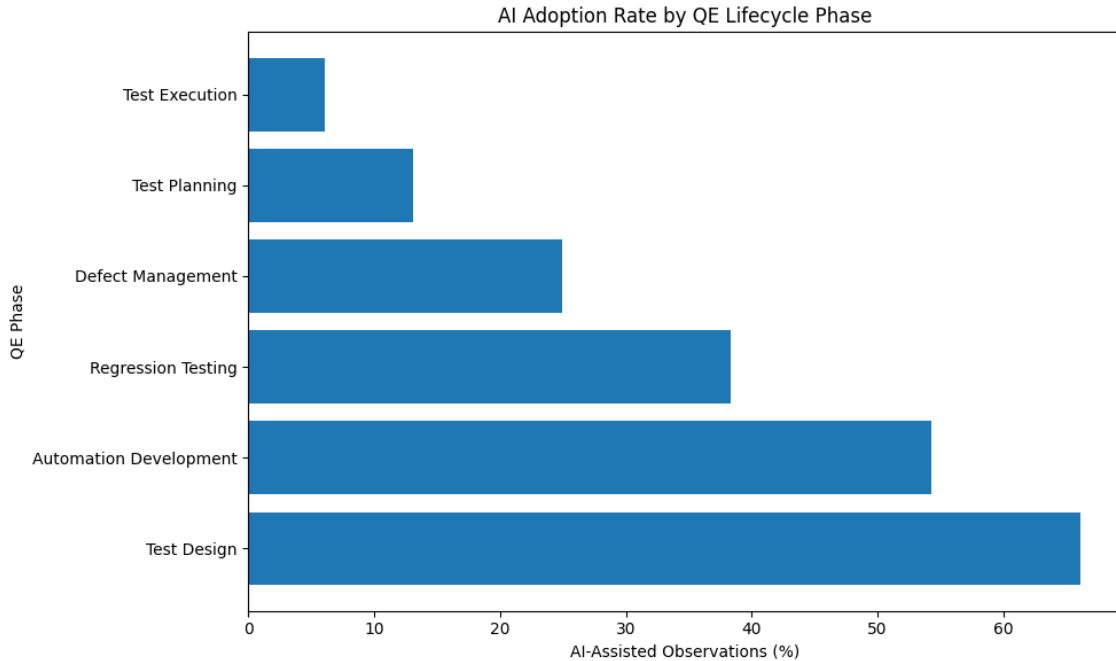


Figure 6: *AI Adoption Rate by QE Lifecycle Phase*

Note. Test Design and Automation Development account for the highest adoption rates, while Test Execution remains the least adopted phase.

AI-Assisted Versus Traditional Productivity

The central exploratory comparison shows a substantial gap between AI-assisted and traditional work. Across 906 valid AI-assisted phase observations, mean productivity improvement was 16.95% and median improvement was 20.26%. Across 1,799 traditional observations, mean improvement was -4.63% and median improvement was -2.71%. The AI-assisted group saved 4,417.6 hours in aggregate, whereas traditional observations exceeded baseline effort by 1,184.1 hours. Figure 7 shows that the distributions overlap, so AI does not guarantee a positive outcome in every case, but the centre of the AI-assisted distribution is clearly higher. The gap is later tested formally under RQ1.

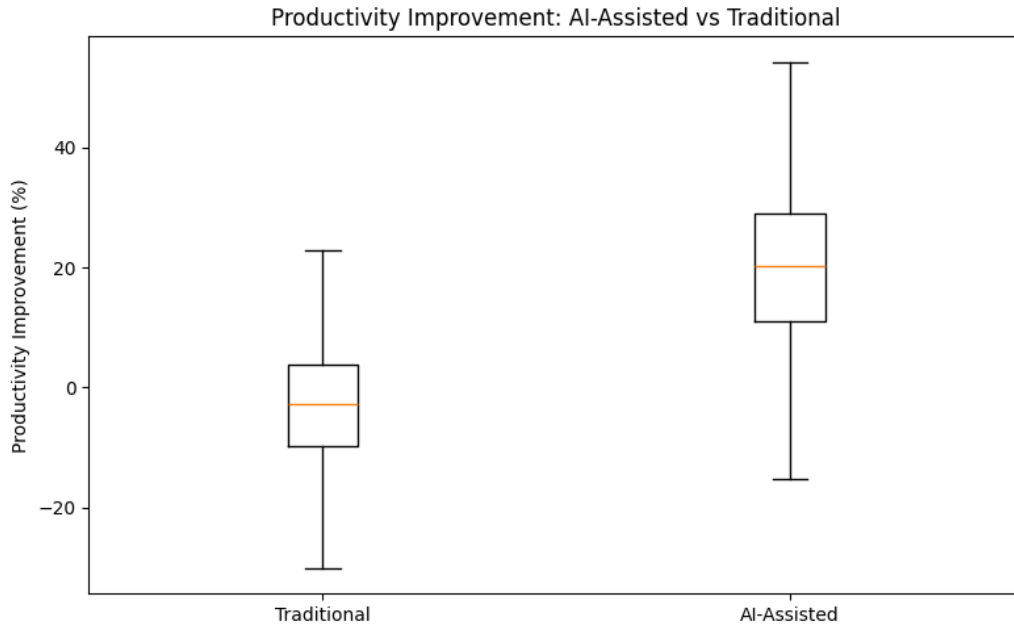


Figure 7: *Productivity Improvement for AI-Assisted and Traditional Activities*

Note. The boxplot suppresses extreme points for visual clarity; all valid observations remain in the statistical analysis.

Productivity and Business Value by Phase

Phase-level analysis indicates that AI value is not uniform. Regression Testing produced the highest mean improvement at 22.23%, followed by Test Design at 19.92% and Automation Development at 16.62%. Defect Management averaged 10.27%, Test Planning 6.46%, and Test Execution 3.93%. Cumulative effort savings tell a complementary story: Test Design saved approximately 1,857.6 hours and Automation Development 1,420.2 hours because they combine substantial improvement with larger adoption volume. Regression Testing saved 896.6 hours despite fewer AI-assisted observations. Figure 8 therefore supports a two-dimensional prioritisation approach: percentage improvement indicates efficiency, while total hours saved indicates portfolio-scale impact.

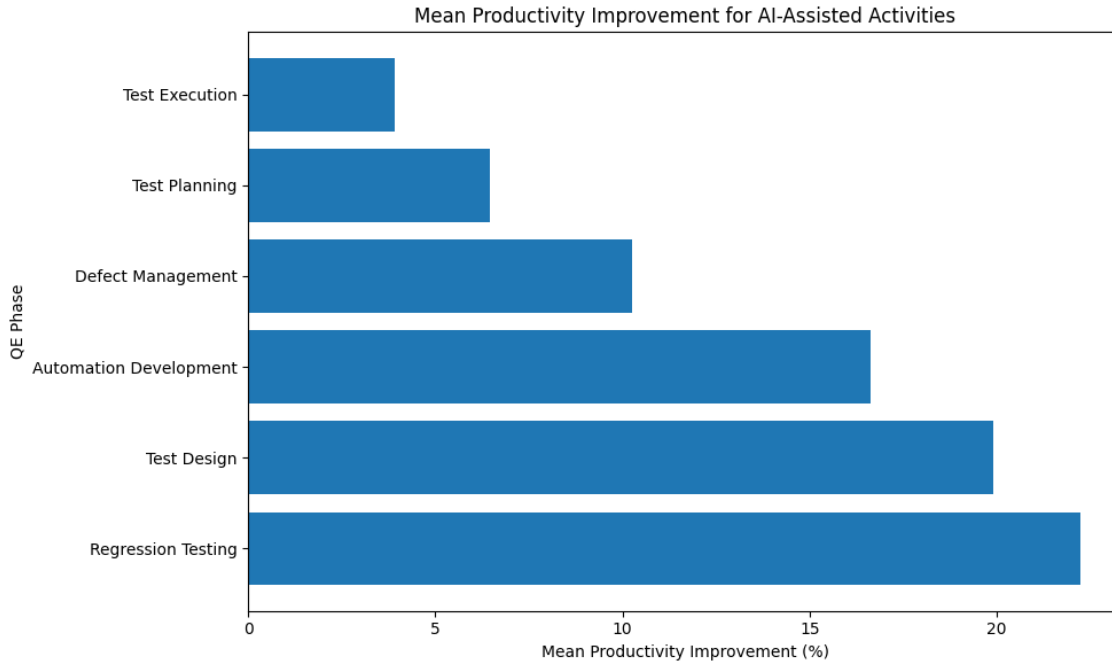


Figure 8: *Mean Productivity Improvement for AI-Assisted Activities by Phase*

Note. Regression Testing has the highest mean percentage improvement; Test Design and Automation Development also demonstrate strong preliminary value.

Breadth of AI Adoption

Overall project/sprint productivity increased as AI was used across more lifecycle phases.

Records with no AI averaged -4.62% overall productivity, while projects using AI in one, two, three, and four phases averaged 0.98% , 6.89% , 9.73% , and 12.97% , respectively. Projects with five phases averaged 16.93% , and the small group with all six phases averaged 18.47% . Figure 9 suggests that adoption breadth is a useful portfolio signal, but it should not be read as causal proof that adding AI to every phase automatically creates improvement. More mature teams may both adopt AI broadly and manage delivery more effectively. This is why AI Phase Count is treated as a predictive feature and the final recommendations retain phase-specific governance.

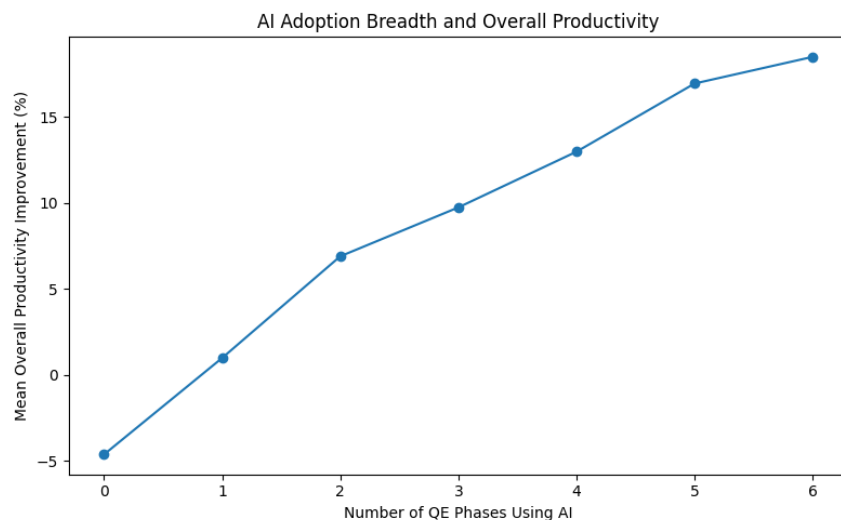


Figure 9: Relationship Between AI Adoption Breadth and Overall Productivity

Note. The six-phase group contains only seven records, so its mean is less stable than estimates for larger groups.

Figure/Table Reference	What It Shows	Key Insight	Why It Matters	Decision/Next Step
Figure 5	AI adoption by year	Adoption rose from 7.17% in 2023 to 64.20% in 2026	Time trend may influence RQ1/RQ3	Retain year and sprint controls
Figure 6	Adoption by phase	Test Design and Automation Development lead; Test Execution lags	Supports phase-specific RQ2 analysis	Avoid uniform rollout assumptions
Figure 7	AI vs traditional productivity	AI group centre is materially higher than traditional	Direct preliminary evidence for RQ1	Use Mann–Whitney and effect sizes
Figure 8	AI productivity by phase	Regression Testing, Test Design, and Automation Development lead	Identifies likely priority areas for RQ2	Run Kruskal–Wallis and post-hoc tests
Figure 9	AI breadth and overall productivity	Wider adoption is associated with stronger overall results	Provides candidate feature for RQ4	Include AI Phase Count and exposure features

Table 7: EDA Insight Summary

Modelling

Choice of Models With Justification

The modelling stage addresses two distinct analytical needs. First, a multiple linear regression at the phase level is used to estimate the relationship between AI usage and productivity while controlling for phase, application, complexity, baseline effort, workload, year, and sprint.

Cluster-robust standard errors are calculated by sprint because six phase observations can originate from the same project/sprint record. This model is primarily explanatory: the AI coefficient can be interpreted in percentage points after other included factors are held constant. Second, three supervised regression algorithms are compared at the project/sprint level for RQ4. Linear Regression is included as an interpretable baseline and to test whether the relationship is approximately additive. Random Forest Regression is included because project outcomes may depend on nonlinear thresholds and interactions among AI breadth, baseline workload, phase allocation, complexity, and application context. Gradient Boosting Regression is included because sequentially correcting residual errors often performs well on structured tabular data. Comparing these models provides a more defensible result than reporting only the highest-performing algorithm.

The validation design is intentionally grouped by sprint. An 80/20 holdout split assigns complete sprint groups to either training or test data so that records from the same delivery period do not appear on both sides. Grouped five-fold cross-validation is then used to assess stability. This is stricter than a random row split and reduces information leakage caused by shared timing, organisational context, or portfolio conditions. The target is winsorised only in the predictive dataset at the 1st and 99th percentiles; actual effort and all variables derived from actual outcomes are excluded from the feature set.

Features Included and Feature Engineering

Feature	Original/Engineered	Type	Reason for Inclusion	Used in Model(s)
Application_Name	Original	Categorical	Captures platform/context differences	All predictive models
Team_Name	Original	Categorical	Captures organisational context; reviewed for potential overfitting	All predictive models
Complexity	Original	Categorical	Represents assessed delivery difficulty	All predictive models
User story count	Original	Numeric	Workload size proxy	All predictive models
test case count	Original	Numeric	Testing volume/intensity proxy	All predictive models
Baseline effort by phase	Original	Numeric	Expected workload distribution	All predictive models
Total_Baseline_Effort	Engineered	Numeric	Overall project effort scale	All predictive models
AI indicator by phase	Original recoded	Binary	Identifies where AI is applied	All predictive models
AI_Phase_Count	Engineered	Integer 0–6	Measures breadth of adoption	All predictive models
AI_Adoption_Rate	Engineered	Proportion	Normalises adoption breadth	Sensitivity/alternative specification
Baseline Share by Phase	Engineered	Proportion	Represents workload allocation across lifecycle	All predictive models
AI Exposure by Phase	Engineered	Proportion	Weights AI use by phase workload	All predictive models
Story-to-Test Ratio	Engineered	Numeric	Proxy for test intensity per requirement	All predictive models
Year and Sprint Number	Engineered	Numeric	Controls temporal adoption/maturity patterns	Influence model; candidate predictive controls
Actual effort / effort saved	Outcome-derived	Excluded	Would reveal the target and create leakage	None

Table 8: Feature Set and Usage

Note. Categorical variables are one-hot encoded. Numeric variables are median-imputed; scaling is applied where required by the model pipeline.

Evaluation Metrics (Formulae and Illustrative Calculations)

Metric	Formula	Interpretation	Used For	Illustrative Calculation
Productivity Improvement %	$((\text{Baseline} - \text{Actual}) / \text{Baseline}) \times 100$	Relative effort reduction; positive is favourable	Outcome engineering	Baseline 20, actual 16: $(4/20) \times 100 = 20\%$
MAE	$(1/n) \sum y_i - \hat{y}_i $	Average absolute prediction error in percentage points	Model comparison	Observed 12, predicted 8: absolute error = 4 points
RMSE	$\sqrt{[(1/n) \sum (y_i - \hat{y}_i)^2]}$	Penalises large prediction errors more strongly	Model comparison	Errors 2 and 6: $\sqrt{[(4+36)/2]} = 4.47$
R ²	$1 - [\sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2]$	Share of unseen target variance explained	Model comparison	R ² = .405 means 40.5% of test variation explained
Cohen's d	$(\text{Mean}_1 - \text{Mean}_2) / \text{pooled SD}$	Standardised practical difference between two means	RQ1 effect size	$d \approx 1.16$ indicates a large difference
Rank-biserial correlation	$2U/(n_1 n_2) - 1$ (direction depends on group order)	Non-parametric effect size for Mann–Whitney	RQ1 robustness	$ r $ near .71 indicates strong separation

Table 9: *Evaluation Metrics and Formulae*

Note. Only regression and statistical-comparison metrics relevant to this project are retained; classification metrics such as accuracy, precision, recall, and F1-score are not applicable

Preliminary Results

Preliminary Findings by Research Question

RQ1: Does AI Adoption Significantly Improve QE Productivity?

Preliminary evidence strongly supports a productivity difference. AI-assisted phase observations averaged 16.953% improvement, compared with -4.629% for traditional observations, a difference of 21.582 percentage points. Because the distribution contained outliers and Levene's test indicated unequal variances, the Mann–Whitney U test was selected as the primary comparison. The result was $U = 1,392,443$, $p < .001$. Welch's t-test produced a consistent sensitivity result, $t = 27.481$, $p < .001$. Cohen's d was 1.159 and the rank-biserial correlation was approximately 0.709, indicating that the difference is not only statistically significant but also practically large. At the interim stage, H_{01} is rejected, subject to final documentation and sensitivity review.

RQ2: Does Business Value Differ Across QE Lifecycle Phases?

The preliminary Kruskal–Wallis test across AI-assisted phases produced $H = 92.721$, $p < .001$, with epsilon-squared = .097. This indicates that phase membership explains a meaningful portion of the rank variation and that the six phases should not be treated as equivalent. Holm-adjusted pairwise Mann–Whitney tests were significant in 13 of 15 comparisons. Regression Testing, Test Design, and Automation Development form the strongest group by mean improvement; Test Planning and Test Execution form the weakest. Test Design and Regression Testing were not significantly different after adjustment, and Test Planning and Test Execution were also not significantly different. H_{02} is therefore rejected at the interim stage.

RQ3: Which Factors Influence Productivity Improvement?

A multiple linear regression was estimated using the valid phase-level dataset. The productivity-improvement outcome was winsorised at the 1st and 99th percentiles to reduce the influence of extreme values. The model included AI usage, baseline effort, user-story count, test-case count,

year, sprint number, lifecycle phase, complexity, and application as predictors. Sprint-clustered robust standard errors were used to account for potential dependence among observations belonging to the same sprint.

AI usage showed the strongest statistical association with productivity improvement among the coefficients reported in the model. After controlling for the other included variables, AI usage was associated with a 16.686-percentage-point increase in productivity improvement, $B = 16.686$, 95% CI [14.247, 19.125], $p < .001$. Test-case count also had a statistically significant positive association with productivity improvement, $B = 0.050$, 95% CI [0.014, 0.086], $p = .006$.

Lifecycle phase was also associated with productivity improvement. Relative to the omitted phase reference category, Test Execution was associated with lower productivity improvement, $B = -6.223$, 95% CI [-10.049, -2.398], $p = .001$. Defect Management was also associated with lower productivity improvement, $B = -4.673$, 95% CI [-7.592, -1.755], $p = .002$, as was Test Planning, $B = -6.144$, 95% CI [-9.984, -2.304], $p = .002$. In contrast, Test Design was associated with higher productivity improvement, $B = 3.077$, 95% CI [0.763, 5.391], $p = .009$.

Among the application categories, Customer Web Portal had a statistically significant positive coefficient relative to the omitted application reference category, $B = 2.563$, 95% CI [0.133, 4.992], $p = .039$. The coefficients for baseline effort, user-story count, year, low complexity, Enterprise Data Platform, Mobile Service App, and CRM Sales & Service were not statistically significant at the .05 level.

These results represent conditional associations within the fitted regression model and should not be interpreted as evidence of causal effect.

RQ4: Can Models Predict Improvement and Support Prioritisation?

Random Forest produced the strongest preliminary grouped holdout performance, with MAE = 5.685 percentage points, RMSE = 7.286, and $R^2 = .405$. Linear Regression was close, with MAE = 5.802, RMSE = 7.460, and $R^2 = .376$. Gradient Boosting produced MAE = 5.935, RMSE = 7.512, and $R^2 = .368$.

Grouped five-fold cross-validation also ranked Random Forest first, with mean CV $R^2 = .471$ and standard deviation = .083. The model therefore provides moderate directional value: an average absolute error near 5.7 percentage points may be acceptable for screening a portfolio, but it is too large for precise budgeting or automatic commitments. Preliminary support is provided for H_{14} , with the qualification that the model is decision support rather than an autonomous forecast.

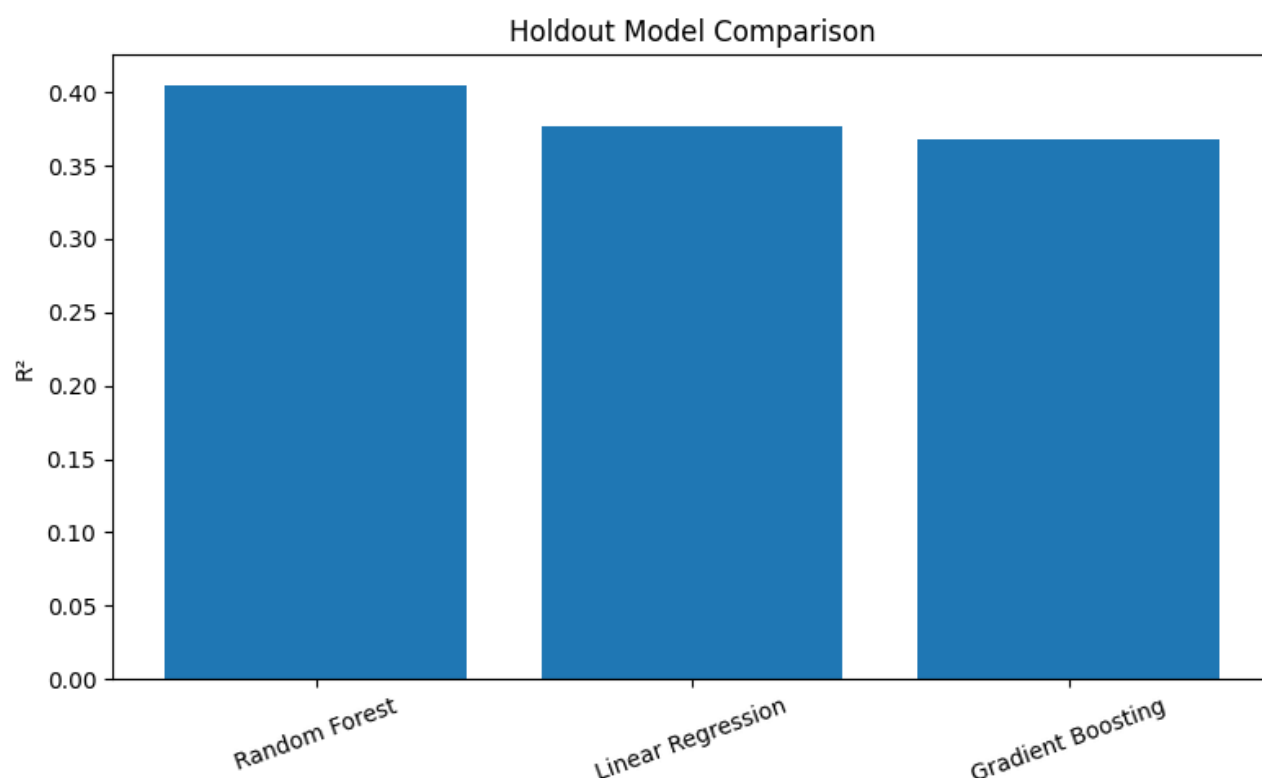


Figure 10: *Grouped Holdout Model Comparison*

Note. Random Forest achieved the highest preliminary R^2 on sprint-grouped holdout data.

Differences among models are moderate, so final selection also considers stability and interpretability.

Model	Feature Set	Validation Approach	Metric 1	Metric 2	Key Takeaway
Random Forest	Project context, baseline workload, AI breadth, phase shares and exposures	80/20 sprint-grouped holdout; grouped 5-fold CV	MAE 5.685	R ² .405	Best holdout and CV performance; useful for directional screening
Linear Regression	Same encoded feature set with scaled numeric fields	80/20 sprint-grouped holdout; grouped 5-fold CV	MAE 5.802	R ² .376	Strong transparent baseline; coefficients easier to explain
Gradient Boosting	Same encoded feature set	80/20 sprint-grouped holdout; grouped 5-fold CV	MAE 5.935	R ² .368	Competitive but did not exceed Random Forest in current tuning

Table 10: *Preliminary Model Performance Comparison*

Preliminary Feature Importance

Permutation importance for the selected Random Forest identifies AI Phase Count as the dominant feature, with a mean decrease in R² of approximately .403 when permuted. The next features—baseline Test Planning effort, Test Execution baseline share, and phase-specific AI exposure variables—have much smaller importance values. This result is consistent with the exploratory breadth pattern, but it should not be interpreted as causal proof. AI Phase Count may also proxy programme maturity, standardisation, and leadership support. Final reporting should therefore pair the importance chart with partial dependence or sensitivity checks and should avoid presenting the model as evidence that every project should enable all six phases.

RQ	Work Completed	Preliminary Output	Interim Decision	Remaining Work
RQ1	Completed preliminary test	AI mean 16.95% vs traditional -4.63%; $p < .001$; large effect	Reject H_{01} provisionally	Final sensitivity narrative
RQ2	Completed omnibus and post-hoc tests	$H = 92.721$; $p < .001$; 13/15 adjusted comparisons significant	Reject H_{02} provisionally	Final phase-priority interpretation
RQ3	Completed preliminary influence model	AI coefficient +16.686 points; $p < .001$; model $R^2 .272$	Reject H_{03} provisionally	Check specification stability
RQ4	Baseline models completed	Random Forest holdout $R^2 .405$, MAE 5.685	Partial support for H_{14}	Final tuning, diagnostics, and usage boundaries

Table 11: *Research Question Progress and Interim Decision*

Interim Limitations and Risks

- The dataset is anonymised and does not contain direct financial cost, licence cost, defect leakage, test coverage, automation coverage, rework, or quality-acceptance measures. Productivity is therefore an incomplete proxy for total business value.
- The analysis is observational. AI-adopting teams may differ from traditional teams in maturity, process discipline, leadership, toolchain, or project selection. Regression controls reduce but do not eliminate confounding.
- AI usage is binary by phase. The data does not distinguish tool type, frequency, assistance intensity, prompt quality, or human-review effort, all of which may explain variation within the AI-assisted group.
- Multiple phase observations originate from the same project/sprint record. Cluster-robust inference and grouped validation address dependence, but a future hierarchical model may represent application, team, sprint, and phase structure more directly.

- Percentage improvement can be unstable when baseline effort is small. Non-parametric tests and winsorisation reduce sensitivity, but final reporting should also show absolute hours saved and minimum baseline thresholds.
- The best predictive model explains approximately 40.5% of unseen variation. Important operational predictors remain unmeasured, and the model should not be used for precise budgets or individual performance assessment.

Next Steps for the Final Report

- Insert the active GitHub repository URL and verify that the evaluator can access all raw, processed, code, and output files.
- Re-run the notebook from a clean environment and record Python and package versions in the README.
- Complete sensitivity checks for alternative outlier treatment, model specifications, and grouped validation seeds.
- Review residual and error distributions and identify project segments where prediction error is materially higher.
- Validate the stability of permutation importance and test whether AI Phase Count remains dominant under alternative feature sets.
- Integrate productivity findings with governance considerations from the literature, especially human validation and output-quality controls.
- Finalise the phase-based adoption roadmap: scale now, controlled expansion, and targeted pilots.

- Complete APA 7 checks for headings, table/figure references, citations, bibliography, page numbering, and appendix cross-references.
- Prepare the final report and archive the immutable submission package in the repository.

The final report will retain the distinction between statistical association, predictive usefulness, and causal evidence. Recommendations will be framed as phased decisions supported by observed productivity and operational context, not as a claim that AI should be applied uniformly to every QE activity. Code used to calculate the core productivity measures and model metrics is summarised in Appendix A, and the complete field-level data dictionary is provided in Appendix B.

Bibliography

- 1) Ardic, B., Le Dilavrec, Q., & Zaidman, A. (2026). How students use generative AI for software testing: An observational study. *Empirical Software Engineering*, 31, Article 167.
<https://doi.org/10.1007/s10664-026-10898-0>
- 2) Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
<https://doi.org/10.1023/A:1010933404324>
- 3) Durelli, V. H. S., Durelli, R. S., Borges, S. S., Endo, A. T., Eler, M. M., Dias, D. R. C., & Guimarães, M. de P. (2019). Machine learning applied to software testing: A systematic mapping study. *IEEE Transactions on Reliability*, 68(3), 1189–1212.
<https://doi.org/10.1109/TR.2019.2892517>

- 4) Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232. <https://doi.org/10.1214/aos/1013203451>
- 5) Garousi, V., Felderer, M., Kuhrmann, M., Herkiloğlu, K., & Eldh, S. (2020). Exploring the industry's challenges in software testing: An empirical study. *Journal of Software: Evolution and Process*, 32(8), e2251. <https://doi.org/10.1002/smr.2251>
- 6) Haldar, S., & Capretz, L. F. (2026). Automated test plan generation using large language models. *Software Quality Journal*, 34, Article 8. <https://doi.org/10.1007/s11219-026-09744-9>
- 7) James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning: With applications in R* (2nd ed.). Springer. <https://doi.org/10.1007/978-1-0716-1418-1>
- 8) Ouédraogo, W. C., Kaboré, A. K., Li, Y., Tian, H., Koyuncu, A., Klein, J., Lo, D., & Bissyandé, T. F. (2026). Prompt engineering in LLMs for automated unit test generation: A large-scale study. *Empirical Software Engineering*, 31, Article 103. <https://doi.org/10.1007/s10664-026-10840-4>
- 9) Raharjana, I. K., Husna, O. I., Hariyanti, E., & Fauzi, S. S. M. (2025). Mapping the landscape of deep learning in software testing: A bibliometric analysis. *Discover Artificial Intelligence*, 5(1), Article 402. <https://doi.org/10.1007/s44163-025-00596-z>
- 10) Romero, J. R., Ramírez, A., & García-Martínez, C. (2026). Automated machine learning for test case prioritisation. *Empirical Software Engineering*, 31, Article 120. <https://doi.org/10.1007/s10664-026-10862-y>
- 11) Venugopal, S. (2026). Enterprise Quality Engineering Performance Dataset (EQEPD) https://github.com/satisho/QM640_Data-Analytics_Capstone
- 12) Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., & Wang, Q. (2024). Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering*, 50(4), 911–936. <https://doi.org/10.1109/TSE.2024.3368208>

- 13) Willmott, C. J., & Matsuura, K. (2005). Advantages of the mean absolute error over the root mean square error in assessing average model performance. *Climate Research*, 30, 79–82.
<https://doi.org/10.3354/cr030079>
- 14) Yoo, S., & Harman, M. (2012). Regression testing minimization, selection and prioritization: A survey. *Software Testing, Verification and Reliability*, 22(2), 67–120.
<https://doi.org/10.1002/stvr.430>

Appendix A: Core Analytical Calculations

The executed Jupyter notebook in the GitHub repository contains the complete code. The extracts below document the principal calculations referenced in the report and are included to support reproducibility without reproducing the entire notebook.

```
# Phase-level effort and productivity
```

```
phase_data['Effort_Saved'] = phase_data['Baseline_Effort'] - phase_data['Actual_Effort']
```

```
phase_data['Productivity_Improvement_Pct'] = ( phase_data['Effort_Saved'] /  
phase_data['Baseline_Effort'] * 100)
```

```
# Project-level target
```

```
project_data['Total_Effort_Saved'] = ( project_data['Total_Baseline_Effort'] -  
project_data['Total_Actual_Effort'])
```

```
project_data['Overall_Productivity_Improvement_Pct'] = ( project_data['Total_Effort_Saved'] /  
project_data['Total_Baseline_Effort'] * 100)
```

```
# Leakage-aware grouping concept
```

```
# Split by Sprint # so the same sprint never appears in both train and test data.
```


Appendix B: Complete Field-Level Data Dictionary

Variable Name	Definition	Datatype	Allowed Values/Range	Missing Values Handling	Notes
Sprint #	Project delivery sprint identifier	Text	YYYY-Snn	Not imputed	Used to derive Year and Sprint_Number; grouping key
Application_Name	Anonymised application/platform	Categorical	8 standardised values	Retained as category	One-hot encoded
Team_Name	Anonymised QE team	Categorical	8 standardised values	Retained as category	One-hot encoded / context
Complexity	Project complexity	Categorical	Low, Medium, High	Aliases standardised	One-hot encoded
User story count	Number of user stories	Numeric count	Non-negative	Median imputed	Workload predictor
test case count	Number of test cases	Numeric count	Non-negative	Median imputed	Testing-volume predictor
AI Usage Testing Phase (Test Planning)	AI assistance used in Test Planning	Binary category	Yes/No	Variants standardised	Converted to binary flag
AI Usage Testing Phase (Test Design)	AI assistance used in Test Design	Binary category	Yes/No	Variants standardised	Converted to binary flag
AI Usage Testing Phase (Automation Development)	AI assistance used in Automation Development	Binary category	Yes/No	Variants standardised	Converted to binary flag
AI Usage Testing Phase (Test Execution)	AI assistance used in Test Execution	Binary category	Yes/No	Variants standardised	Converted to binary flag
AI Usage Testing Phase (Regression Testing)	AI assistance used in Regression Testing	Binary category	Yes/No	Variants standardised	Converted to binary flag
AI Usage Testing Phase (Defect Management)	AI assistance used in Defect Management	Binary category	Yes/No	Variants standardised	Converted to binary flag
Baseline effort (Test Planning)	Expected effort for Test Planning in hours	Numeric continuous	Normally > 0	Not imputed for outcomes	Target denominator
Baseline effort (Test Design)	Expected effort for Test Design in hours	Numeric continuous	Normally > 0	Not imputed for outcomes	Target denominator
Baseline effort (Automation Development)	Expected effort for Automation Development in hours	Numeric continuous	Normally > 0	Not imputed for outcomes	Target denominator

Variable Name	Definition	Datatype	Allowed Values/Range	Missing Values Handling	Notes
Baseline effort (Test Execution)	Expected effort for Test Execution in hours	Numeric continuous	Normally > 0	Not imputed for outcomes	Target denominator
Baseline effort (Regression Testing)	Expected effort for Regression Testing in hours	Numeric continuous	Normally > 0	Not imputed for outcomes	Target denominator
Baseline effort (Defect Management)	Expected effort for Defect Management in hours	Numeric continuous	Normally > 0	Not imputed for outcomes	Target denominator
Actual effort (Test Planning)	Observed effort for Test Planning in hours	Numeric continuous	≥ 0	Not imputed for outcomes	Used only to create outcomes; excluded from predictors
Actual effort (Test Design)	Observed effort for Test Design in hours	Numeric continuous	≥ 0	Not imputed for outcomes	Used only to create outcomes; excluded from predictors
Actual effort (Automation Development)	Observed effort for Automation Development in hours	Numeric continuous	≥ 0	Not imputed for outcomes	Used only to create outcomes; excluded from predictors
Actual effort (Test Execution)	Observed effort for Test Execution in hours	Numeric continuous	≥ 0	Not imputed for outcomes	Used only to create outcomes; excluded from predictors
Actual effort (Regression Testing)	Observed effort for Regression Testing in hours	Numeric continuous	≥ 0	Not imputed for outcomes	Used only to create outcomes; excluded from predictors
Actual effort (Defect Management)	Observed effort for Defect Management in hours	Numeric continuous	≥ 0	Not imputed for outcomes	Used only to create outcomes; excluded from predictors
Year	Calendar year derived from Sprint #	Integer	2023–2026	Derived	Trend/control feature
Sprint_Number	Numeric sprint sequence derived from Sprint #	Integer	Positive	Derived	Trend/control feature
QE_Phase	Lifecycle phase in long-format data	Categorical	Six phases	Derived	Phase comparison
AI_Used	Standardised phase-level AI indicator	Binary category	Yes/No	Derived from phase field	RQ1 grouping
AI_Used_Flag	Numeric phase-level AI indicator	Binary numeric	0/1	Derived	Regression input
Baseline_Effort	Phase baseline effort in long data	Numeric continuous	> 0 for target	No imputation	Phase workload

Variable Name	Definition	Datatype	Allowed Values/Range	Missing Values Handling	Notes
Actual_Effort	Phase actual effort in long data	Numeric continuous	≥ 0	No imputation	Outcome input only
Effort_Saved	Baseline_Effort – Actual_Effort	Numeric continuous	Positive/negative	Derived	Absolute benefit
Productivity_Improvement_Pct	Effort_Saved / Baseline_Effort $\times 100$	Numeric continuous	Can be positive/negative	Derived; winsorised only in selected model use	RQ1–RQ3 target
Total_Baseline_Effort	Sum of six phase baselines	Numeric continuous	Positive	Derived	Project scale
Total_Actual_Effort	Sum of six phase actuals	Numeric continuous	Non-negative	Derived	Outcome input; excluded from model features
Total_Effort_Saved	Total baseline – total actual	Numeric continuous	Positive/negative	Derived	Project absolute benefit
Overall_Productivity_Improvement_Pct	Total saved / total baseline $\times 100$	Numeric continuous	Can be positive/negative	Derived; winsorised 1st/99th for models	RQ4 target
AI_Phase_Count	Count of phase AI flags	Integer	0–6	Derived	Adoption breadth
AI_Adoption_Rate	AI_Phase_Count / 6	Numeric proportion	0–1	Derived	Normalised breadth
Baseline_Share_<Phase>	Phase baseline / total baseline	Numeric proportion	0–1	Derived	Workload allocation
AI_Exposure_<Phase>	AI phase flag $\times$ baseline share	Numeric proportion	0–1	Derived	Workload-weighted AI use
Story_to_Test_Ratio	test case count / (user story count + 1)	Numeric continuous	≥ 0	Derived	Test intensity proxy

Table B1: Complete Raw and Engineered Data Dictionary

Note. Angle-bracket notation indicates a repeated engineered field for each of the six QE phases. The notebook contains the exact column names exported to the processed files.

Appendix C: Interim Submission Checklist

Requirement	Status
Title page and page numbers	Complete
Standalone introduction and problem statement	Complete
GitHub link immediately after title page	Complete
Four research questions	Complete
Minimum 10 literature sources	Complete—12 sources in matrix; 16 bibliography entries
Dataset source, overview, and access paths	Complete
Mandatory data dictionary	Summary in main report; full dictionary in Appendix B
Data cleaning log	Complete
EDA figures with interpretation	Complete
Model justification and feature table	Complete
Evaluation metrics with formulae/calculations	Complete
Preliminary results by RQ	Complete
Limitations and next steps	Complete
APA 7 final check	Complete

Table C1: *Interim Report Requirement Checklist*